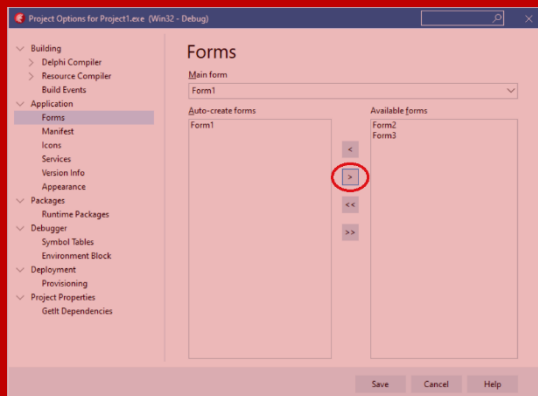
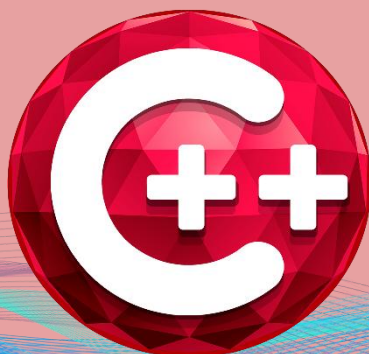




RAD Studio | C++Builder 13

FLORENCE



MÓDULO 1 - BÁSICO

FORMULÁRIOS

C++ BUILDER



Criado por: Thiago Santos

MVP Embarcadero. 2026

SUMÁRIO

Sumário	2
Introdução ao Curso de C++Builder 13 Florence	3
O que você vai aprender neste curso.....	4
Sobre o instrutor.....	5
O que você verá:	6
Introdução.....	7
Menu Principal.....	7
Caixas de Diálogo	18
Herança de formulários	23

Seja bem-vindo!!

Introdução ao Curso de C++Builder 13 Florence

Este curso foi desenvolvido para apresentar o **C++Builder 13 Florence**, a versão mais recente da plataforma de desenvolvimento da Embarcadero, com foco em quem deseja aprender **C++ de forma prática, produtiva e aplicada ao desenvolvimento de aplicações reais**.

Ao longo desta formação, o aluno será introduzido aos principais conceitos do **C++ funcional aplicado ao C++Builder**, entendendo não apenas a linguagem, mas também como utilizá-la de forma eficiente dentro de um ambiente RAD (Rapid Application Development), que acelera o processo de criação de software sem *abrir mão de desempenho e controle*.

A importância de utilizar o C++Builder 13 Florence

Utilizar a versão mais recente do C++Builder é fundamental para garantir **compatibilidade com tecnologias atuais**, maior estabilidade da IDE e acesso aos recursos mais modernos da linguagem C++. O C++Builder 13 Florence traz melhorias significativas no compilador baseado em Clang, suporte aprimorado ao padrão moderno do C++, além de uma IDE mais rápida, segura e alinhada às necessidades do mercado atual.

Aprender com uma versão atual evita retrabalho futuro, reduz problemas de compatibilidade e prepara o aluno para atuar em projetos profissionais, corporativos e comerciais, utilizando ferramentas reconhecidas e amplamente adotadas por empresas.

O que você vai aprender neste curso

Este curso tem como objetivo fornecer uma **base funcional e prática**, permitindo que o aluno desenvolva aplicações reais desde os primeiros módulos. Entre os principais conhecimentos adquiridos, destacam-se:

- Fundamentos do C++ aplicados ao C++Builder
- Estrutura da IDE e fluxo de desenvolvimento
- Criação de aplicações visuais utilizando VCL
- Manipulação de eventos e componentes
- Lógica de programação aplicada a projetos reais
- Integração básica com banco de dados
- Boas práticas para desenvolvimento em C++Builder

Ao final do curso, o aluno terá conhecimento suficiente para **criar aplicações funcionais**, entender projetos existentes e evoluir para níveis mais avançados da linguagem e da ferramenta.

Por que escolher o C++Builder

O C++Builder se diferencia por unir o **alto desempenho do C++** com a **produtividade do desenvolvimento visual**. Ele permite criar aplicações robustas, rápidas e profissionais com muito menos código do que abordagens tradicionais, mantendo total controle sobre a linguagem e o desempenho.

Além disso, o C++Builder é amplamente utilizado em sistemas corporativos, aplicações industriais, softwares comerciais e soluções que exigem **performance, estabilidade e longevidade**. Aprender C++Builder é investir em uma tecnologia madura, sólida e ainda extremamente relevante no mercado.

Este curso é o primeiro passo para quem deseja dominar o **C++ moderno aplicado ao desenvolvimento profissional**, utilizando uma das ferramentas mais completas e produtivas disponíveis atualmente.

Está pronto? Vamos começar!!

Sobre o instrutor

Meu nome é **Thiago Santos** e sou profissional da área de tecnologia, atuando há vários anos com **desenvolvimento de software, análise de negócios e treinamentos técnicos**, com foco na linguagem **C++** e na plataforma **C++Builder**.

Ao longo da minha trajetória profissional, tive a oportunidade de trabalhar diretamente com projetos reais, ambientes corporativos e soluções que exigem **alto desempenho, estabilidade e manutenção a longo prazo**. Essa experiência prática me permitiu entender não apenas a linguagem C++, mas também como aplicá-la de forma eficiente no dia a dia de empresas e equipes de desenvolvimento.

Sou **MVP da Embarcadero na linguagem C++**, reconhecimento concedido a profissionais que contribuem ativamente para a comunidade, compartilham conhecimento e utilizam as tecnologias Embarcadero em projetos reais. Além disso, atuo como **instrutor, agente de negócios e consultor técnico**, auxiliando empresas e desenvolvedores na adoção e evolução do C++Builder em seus sistemas.

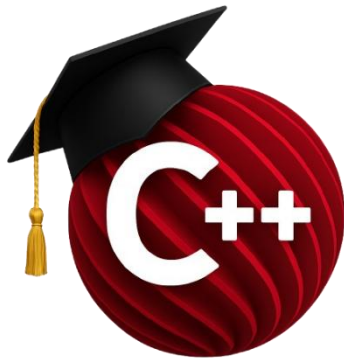
Neste curso, meu objetivo não é apenas ensinar comandos ou conceitos isolados, mas **transmitir a forma correta de pensar, estruturar e desenvolver aplicações em C++Builder**, com base em experiências reais de mercado. Todo o conteúdo foi planejado para ser direto, prático e aplicável, evitando excesso de teoria desconectada da prática.

Acredito que aprender programação vai muito além de escrever código: envolve entender processos, tomar decisões técnicas corretas e criar soluções que realmente funcionem. É exatamente essa visão que compartilho ao longo deste curso.

Seja bem-vindo(a) à formação. Espero que este conteúdo contribua de forma significativa para sua evolução profissional e técnica no desenvolvimento com **C++Builder**.

Meus contatos e plataformas estão disponíveis:

<https://sam-cpp.wordpress.com/>



FORMULÁRIOS

PERFIL

Este treinamento acadêmico visa apresentar ao aluno um ambiente integrado de desenvolvimento de alta produtividade, que permita a criação de aplicações de todos os tipos e para as mais importantes plataformas da atualidade.

Neste capítulo serão apresentados os conceitos e componentes envolvidos no acesso a dados e componentes de manipulação dos mesmos.

Caso queira se aprofundar e se especializar no conhecimento da ferramenta, sua linguagem, tecnologias e recursos, uma lista de centros de treinamentos oficiais está disponível:

www.embarcadero.com.br

CONTATO

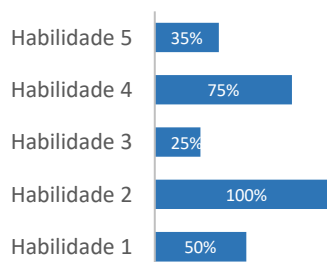
academico@embarcadero.com.br

O que você verá:

Este módulo do treinamento acadêmico é composto por:

- Menu Principal
- Gerenciamento e Chamadas de Formulários
- Caixas de Diálogo
 - Herança de Formulários

HABILIDADES



INTRODUÇÃO

Uma das características mais importantes no desenvolvimento de uma aplicação é o uso de janelas e componentes visuais para garantir uma boa acessibilidade e interatividade com o usuário. No ambiente de programação, estas janelas ganham o nome de “Formulários”, por se tratar de um recurso que permite a recuperação, manipulação e o processamento de dados inseridos pelo usuário.

O **C++Builder** oferece diversas **funções, classes e propriedades** relacionadas à **modelagem de formulários** em um projeto VCL, como a criação de **menus, caixas de diálogo** e o **gerenciamento de janelas**. Além disso, o C++Builder permite que o desenvolvedor utilize plenamente os **conceitos de Orientação a Objetos em C++**, criando **novos formulários como classes derivadas de TForm**, com suporte a **herança, encapsulamento e polimorfismo**.

As imagens e exemplos a seguir fornecem uma **introdução básica sobre formulários no C++Builder**, sendo suficientes para motivar o **aprofundamento dos estudos na linguagem C++ e na biblioteca VCL**.

.

MENU PRINCIPAL

Um **menu principal** no **C++Builder** consiste em uma **barra superior do formulário**, composta por **títulos (itens de menu)** que identificam sua função ou grupo de funções. O principal objetivo do menu principal é **organizar as funcionalidades da aplicação por assunto e auxiliar o usuário na localização das ações disponíveis**.

Para a criação do menu principal, o **C++Builder disponibiliza o componente TMainMenu**, localizado na **paleta de componentes**, normalmente na categoria **Standard**.

Após arrastar o componente **TMainMenu** para o formulário, o próximo passo é abrir o **Menu Designer** para adicionar os títulos do menu principal. Para isso, basta **clicar com o botão direito** sobre o componente e selecionar **Menu Designer**, ou simplesmente **dar um duplo clique** sobre ele.

O **Menu Designer** é aberto exibindo um **pequeno retângulo selecionado** no canto superior esquerdo. Esse retângulo representa o **primeiro item do menu principal**, cujo texto é definido pela propriedade **Caption**. Por exemplo, ao definir **“Cadastros”** na propriedade **Caption**, o título passa a ser exibido **tanto no Menu Designer quanto diretamente no formulário**.

.

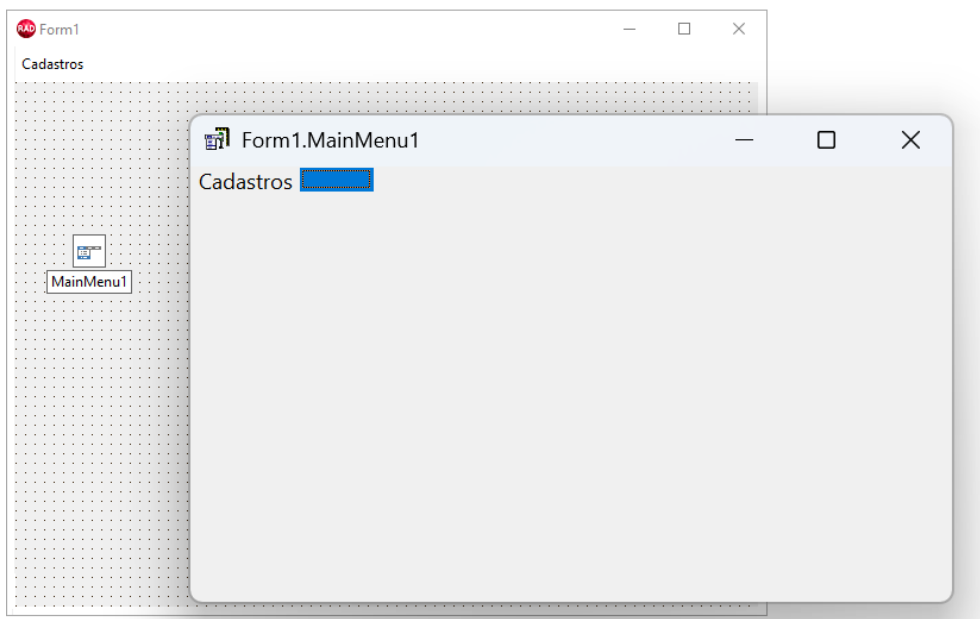


Figura 1: Introdução ao componente MainMenu

A partir deste momento surge duas opções: adicionar um novo menu à direita ou adicionar novos menus abaixo do título “Cadastros”. Em continuidade ao exemplo acima, clique sobre o título “Cadastros” e em seguida no retângulo que é exibido abaixo do título. Neste retângulo, utilize novamente a propriedade *Caption* e digite “Clientes” para que um novo menu seja adicionado, alinhado ao menu principal “Cadastros”. Repita os passos acima e adicione mais dois menus: “Produtos” e “Fornecedores”, conforme a Figura 2:

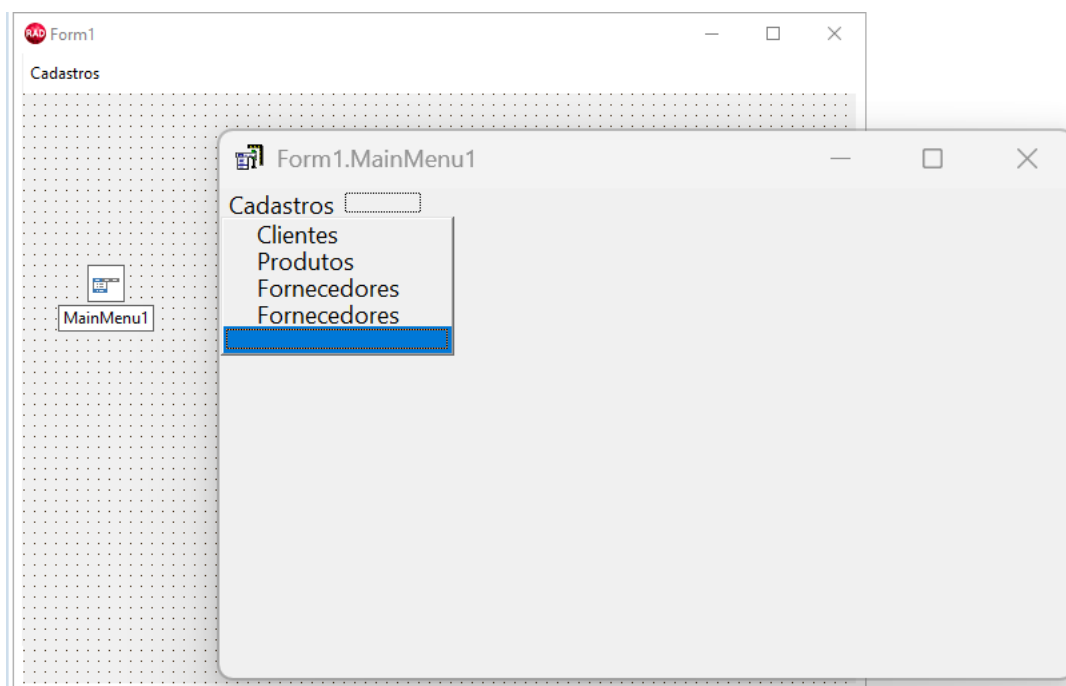


Figura 2: Inserindo novos menus no componente MainMenu

O próximo passo é criar um novo menu principal intitulado “Consultas”. Para isso, basta clicar no retângulo com bordas pontilhadas localizado à direita do menu principal “Cadastros”, e então usar a propriedade *Caption* para digitar “Consultas”. Abaixo deste menu principal, adicione um novo menu com o nome “Vendas”.

Em determinadas ocasiões será necessário adicionar um separador entre os menus para indicar a divisão de funções e facilitar a visibilidade do menu principal. No espaço em branco abaixo do menu “Vendas”, ao invés de digitar uma palavra, digite apenas um hífen (-) na propriedade *Caption*, e então pressione ENTER para adicionar um separador. Em seguida, adicione um novo menu chamado “Pedidos” abaixo deste separador.

Como o menu “Pedidos” será utilizado pelo usuário com grande frequência, talvez seja melhor adicionar uma tecla de atalho para agilizar o acesso à este menu. Para isso, é necessário configurar a propriedade *ShortCut* indicando a tecla de atalho desejada. Selecione a combinação “CTRL + F8” nesta propriedade – toda vez que o usuário teclar esta combinação, o menu será aberto sem a necessidade de utilizar o mouse.

Os menus criados pelo C++Builder também permitem que sejam marcados e desmarcados por um símbolo de checagem à esquerda do menu. Para exemplificar este recurso, adicione um novo menu abaixo de “Pedidos”, nomeie-o como “Verificar Pedidos Duplicados” e em seguida configure as propriedades *AutoCheck* e *Checked* como True. Altere também a propriedade *Name* deste menu para “mnuVerificarDuplicados”, pois este item será referenciado no código-fonte mais à frente. Após todos os exemplos acima, o menu principal do formulário deverá ser semelhante à Figura 3.

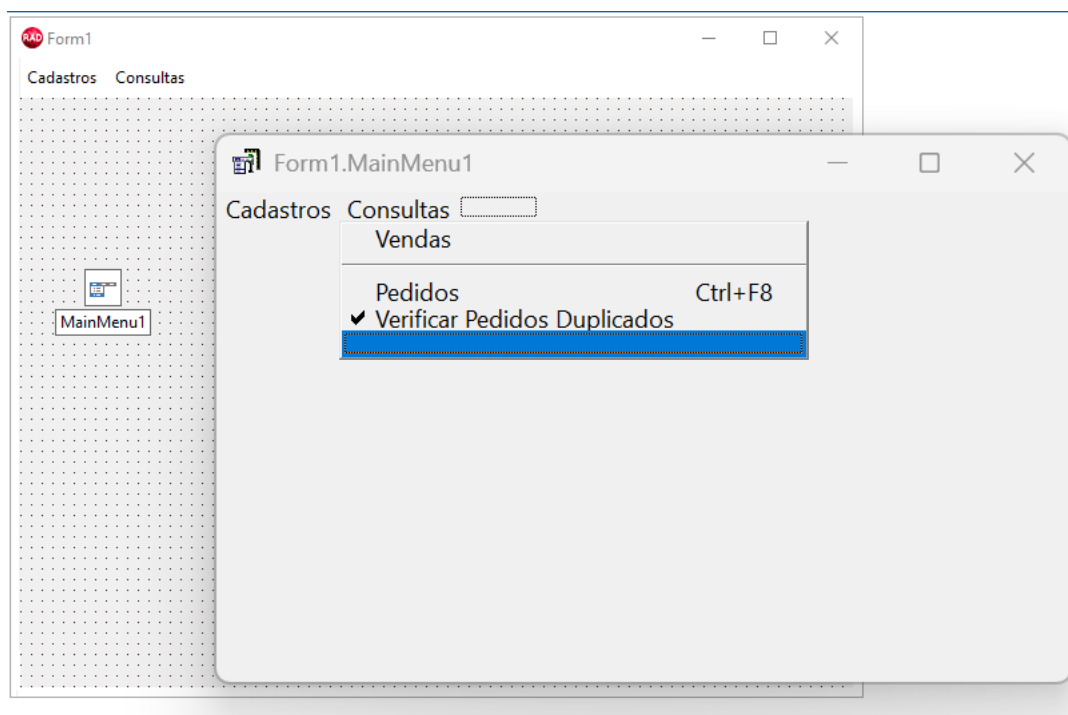


Figura 3: Conhecendo as propriedades do componente MainMenu

É importante salientar que os menus não funcionarão em tempo de projeto, ou seja, para testar os menus criados, é preciso executar a aplicação através do menu *Run*. Em tempo de execução, note que ao clicar no menu “Verificar Pedidos Duplicados”, ele é desmarcado e, ao clicar sobre ele novamente, a marcação volta a aparecer.

Uma grande característica do componente MainMenu é a possibilidade de criar sub-menus, utilizados para agrupar as funções de uma aplicação de uma forma mais detalhada. Por exemplo, no *Menu Designer*, clique sobre o menu “Vendas”, segure a tecla CTRL e pressione a seta para a direita. Este mesmo atalho pode ser acessado clicando com o botão direito sobre o menu “Vendas” e selecionando a opção *Create Submenu*. O procedimento cria um sub-menu à direita do menu “Vendas” indicado por uma pequena seta. Neste sub-menu, por exemplo, adicione os itens “Vendas Por Período” e “Vendas por Cliente”, conforme a Figura 4:

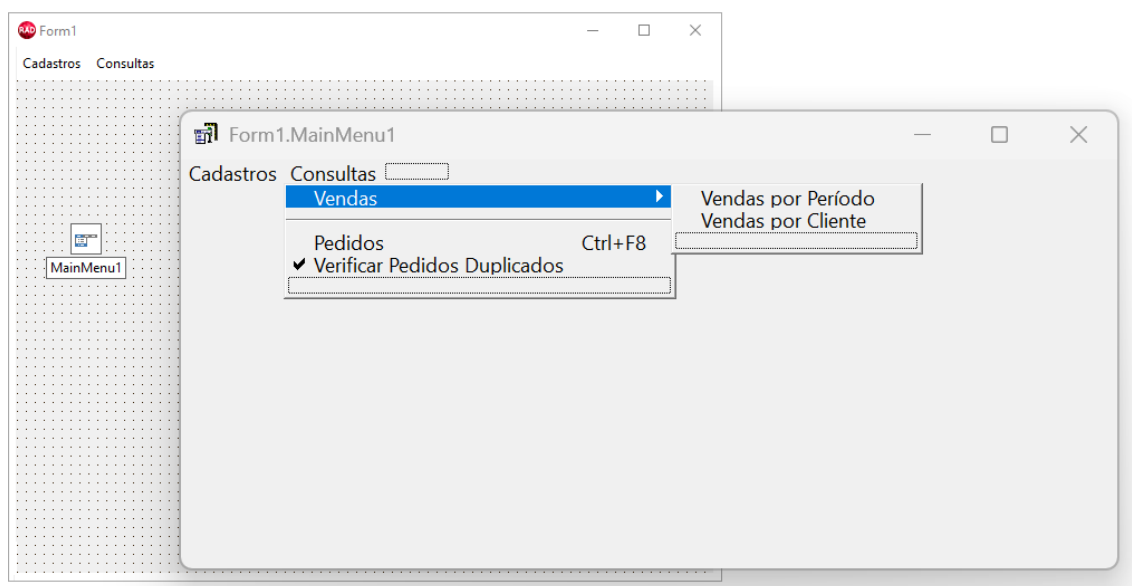


Figura 4: Inserindo sub-menus no componente MainMenu

Como se pode notar, o MainMenu disponibiliza uma gama de recursos para enriquecer o menu principal de uma aplicação. O próximo passo é escrever o código-fonte para que os menus tenham funcionalidade quando o usuário clicar sobre eles. Por exemplo, abra o *Menu Designer* e clique duas vezes no menu “Pedidos”, abaixo do menu principal “Consultas”. O editor de código é exibido no Delphi e o evento de clique no menu é criado automaticamente para que os códigos sejam digitados. Neste exemplo, digite o código abaixo:

```
//-----
void __fastcall TForm1::Pedidos1Click(TObject *Sender)
{
    ShowMessage("O menu Consulta de pedidos foi acionado!");
    if (VerificarPedidosDuplicados1->Checked) {
        ShowMessage("A verificação de pedidos duplicados está vazia");
    }
    else
        ShowMessage("A verificação de pedidos duplicados está inativa!");
}
//-----
```

Código 1

Ao executar a aplicação, verifique que ao pressionar as teclas de atalho “CTRL + F8” a mensagem será exibida da mesma forma como se o menu fosse clicado com o mouse. Experimente também marcar a opção de verificar pedidos duplicados e acionar o menu de consulta novamente – a mensagem será diferente de acordo com o estado do menu.

Gerenciamento e chamada de formulários

Geralmente uma aplicação é constituída de várias janelas (ou formulários) que devem ser abertas conforme a ação do usuário. Dependendo do porte da aplicação desenvolvida, pode existir dezenas ou até centenas de janelas (ou formulários) contidas nesta aplicação. Os exemplos abaixo demonstram como os formulários são criados no C++Builder são exibidos na tela para o usuário.

Quando uma nova aplicação VCL é criada no C++Builder, um formulário já vem adicionado por padrão com o nome de *Form1*. Assim como cada componente possui suas propriedades para configuração, o próprio formulário também conta com algumas propriedades importantes em relação ao seu comportamento:

Propriedade *Caption*: define a descrição que será exibida na barra de títulos do formulário.

Propriedade *BorderIcons*: configura a visibilidade dos botões na parte superior direita do formulário. Por padrão, os botões de minimizar, maximizar e o menu do sistema estarão habilitados, e podem ser desmarcados conforme a necessidade do desenvolvedor.

Propriedade *BorderStyle*: define o comportamento de redimensionamento e o estilo de borda do formulário. A opção *bsSizeable* permite que o usuário redimensione o formulário clicando sobre a borda e movendo-a com o mouse, enquanto *bsSingle* fixa o tamanho do formulário de acordo com os valores definidos em tempo de projeto pelas propriedades *Height* (Altura) e *Width* (Largura). A opção *bsDialog* define o formulário como uma caixa de diálogo, somente com o botão de fechar e sem opções de redimensionamento. Já a opção *bsNone* remove quaisquer bordas do formulário, inclusive o título e os botões superiores.

Propriedade *Position*: configura a posição inicial do formulário na tela do usuário. Entre as opções disponíveis, normalmente é utilizada a opção *poScreenCenter*, que posiciona o formulário no centro da tela quando é exibido. Em alguns casos excepcionais pode ser necessário posicionar o formulário em outros locais da tela, como uma janela de ajuda, por exemplo.

Propriedade *WindowState*: diz respeito ao estado inicial do formulário quando este é criado e exibido ao usuário. As opções *wsMaximized* e *wsMinimized* definem o formulário como Maximizado ou Minimizado, respectivamente. A opção *wsNormal* (padrão) exibe o formulário conforme a altura e largura definidas nas propriedades *Height* e *Width*.

Para adicionar um novo formulário dentro da aplicação em tempo de projeto, basta clicar no menu **File > New > Form – VCL Form C++Builder**, e depois salvá-lo com o nome desejado. Note que agora há dois formulários existentes no mesmo projeto. Para alternar entre estes formulários, clique no botão *View Form* na barra de ferramentas ou pressione a combinação de teclas “Shift + F12” para abrir a janela demonstrada na Figura 5:

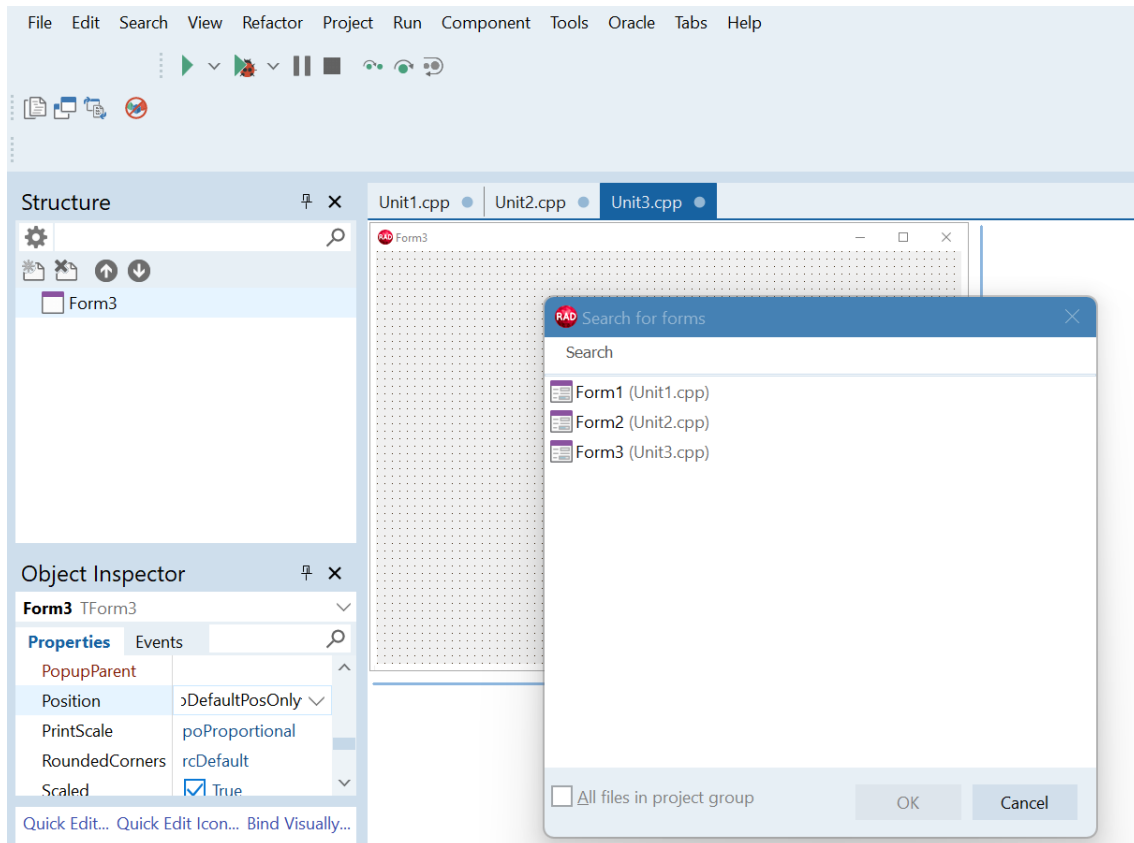


Figura 5: Acessando a lista de formulários do projeto

Na janela de formulários do projeto, selecione o *Form2* e clique no botão OK para que o *Form2* seja exibido na tela. Adicione um botão ao formulário, altere a propriedade *Caption* para “Fechar” e digite o seguinte código:

```

19 //-----
20 void __fastcall TForm2::Button1Click(TObject *Sender)
21 {
22     Close();
23 }
24 //-----

```

Código 2

Este simples comando tem a função de fechar o formulário, semelhante ao efeito de clicar no botão fechar da barra de título. Retorne à janela *View Form* e desta vez acione o *Form1*.

Selecione a **Unit1.cpp** e adicione ao topo do projeto:

```

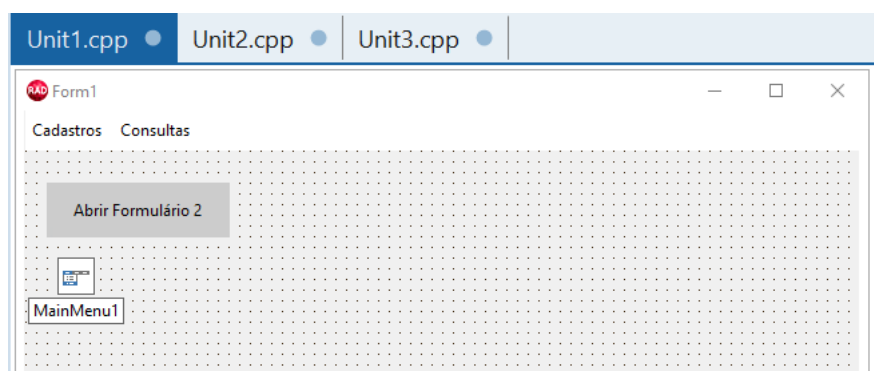
Unit1.cpp | Unit2.cpp | Unit3.cpp |
Search for a type TForm1::Button1Click

#include <vcl.h>
#pragma hdrstop
#include "Unit1.h"
#include "Unit2.h"
//-----
#pragma package(smart_init)
#pragma resource "*.dfm"
TForm1 *Form1;
//-----
__fastcall TForm1::TForm1(TComponent* Owner)
: TForm(Owner)
{
}
//-----
void __fastcall TForm1::Pedidos1Click(TObject *Sender)
{
    ShowMessage("O menu Consulta de pedidos foi acionado!");
    if (VerificarPedidosDuplicados1->Checked) {
        ShowMessage("A verificação de pedidos duplicados está vazia");
    }
    else
        ShowMessage("A verificação de pedidos duplicados está inativa!");
}
//-----

```

Este include “mostra a referência” para o Form1/**Unit1.cpp** que ele vai acessar o **Form2/Unit2.cpp** e suas propriedades.

Agora adicione um botão dentro desse formulário (Form1), altere sua propriedade *Caption* para “Abrir Formulário 2” e digite o código a seguir:



Clicando duas vezes no botão adicionado “Abrir Formulário 2”;

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    Form2->Show();
}
//-----

```

Código 3

Experimente executar a aplicação novamente e verifique que, ao clicar no botão no *Form1*, o *Form2* é exibido na tela, conforme a Figura 7:

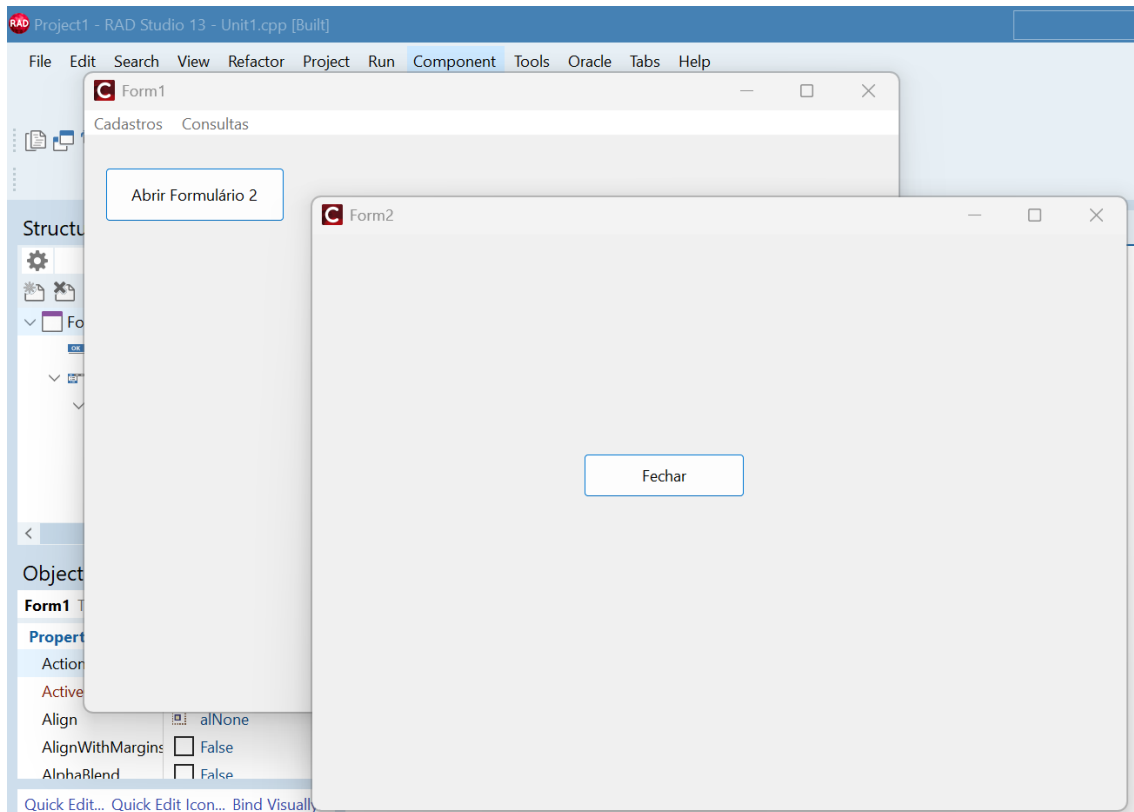


Figura 7: Chamando um formulário em Runtime

No exemplo acima, note que mesmo com o *Form2* aberto, é possível focar novamente o *Form1* sem que o *Form2* seja fechado. Em alguns casos pode ser necessário dar exclusividade ao formulário exibido, mantendo-o focado até que ele seja fechado. Este efeito é conhecido como “Janela Modal”, semelhante ao comportamento de uma caixa de diálogo. Para implementar este recurso ao *Form2*, por exemplo, substitua o código do botão no *Form1* pelo comando abaixo:

```

30 //-----
31 void __fastcall TForm1::Button1Click(TObject *Sender)
32 {
33     Form2->ShowModal();
34 }
35 //-----

```

Código 4

Dessa forma, ao executar a aplicação e abrir o *Form2*, este será exclusivo e não permitirá que o *Form1* seja focado ao menos que o *Form2* seja fechado.

No C++Builder, quando um novo formulário é adicionado a um projeto, este formulário é configurado para ser “carregado” automaticamente toda vez que a aplicação é executada. Quando a aplicação possui poucos formulários, a carga de inicialização é relativamente rápida em virtude destes poucos formulários que serão carregados na memória. Porém, quando uma aplicação contém uma grande quantidade de formulários, torna-se inviável carregá-los automaticamente na inicialização da aplicação. Neste caso,

o C++Builder disponibiliza a possibilidade de criar os formulários sob demanda, ou seja, somente exibi-los quando for necessário.

Em primeiro lugar, é necessário remover os formulários da lista de criação automática. Para isso, acesse o menu **Project > Options** ou pressione a combinação “CTRL + SHIFT + F11”. Na janela *Project Options*, localize o item *Forms* para exibir a lista de formulários. Do lado esquerdo há um painel com os formulários automaticamente criados ao executar a aplicação. O painel do lado direito constitui dos formulários disponíveis na aplicação, mas que deverão ser criados em tempo de execução para serem exibidos. Selecione o item *Form2* e mova-o para a lista de formulários disponíveis clicando no botão indicado pela Figura 8:

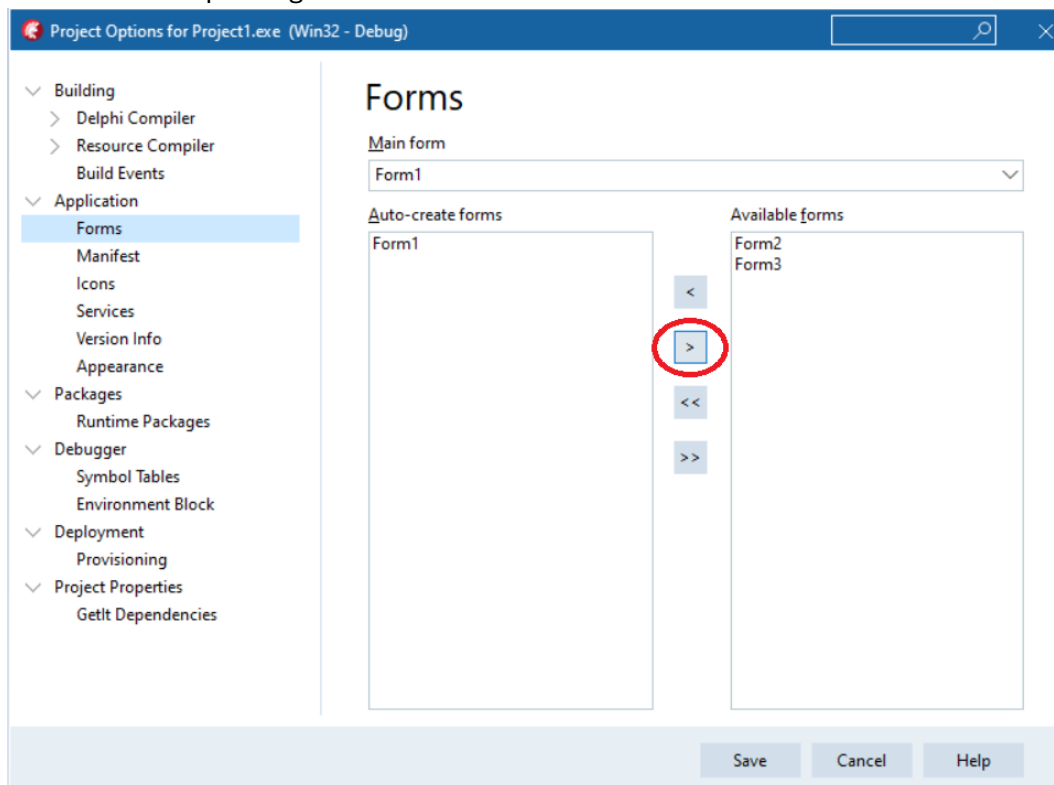
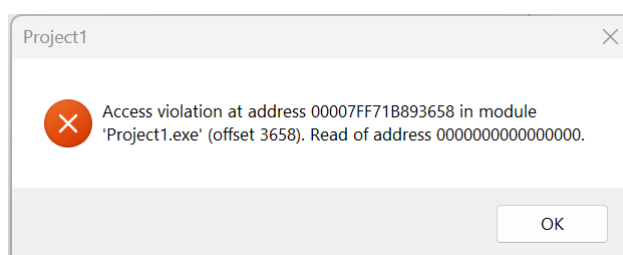


Figura 8: Configurando a criação automática de formulários do projeto

Clique no botão *Save* para fechar a janela. Agora, ao executar a aplicação e tentar abrir o *Form2*, o Delphi levantará uma exceção de violação de acesso. Este tipo de erro ocorre quando o C++Builder tentar fazer referência a um objeto ou recurso que não está disponível atualmente na aplicação. Portanto, para que o formulário seja criado antes de ser exibido, basta adicionar código a seguir no botão “Abrir Formulário 2”:



Erro: Access violation

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    TForm2 *Form2 = new TForm2(this);
    try
    {
        Form2->ShowModal();
    }
    finally
    {
        delete Form2;
    }
}
//-----
```

Código 5

Onde:

TForm2 *Form2 = new TForm2(this); //Cria dinamicamente uma instancia do formulário TForm2, indicando que (this) é o formulário atual.

Para liberar o formulário da memória utilizamos o **delete**. Mas também pode ser utilizado este recurso (**Free**):

```
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    TForm2 *Form2 = new TForm2(this);

    try
    {
        // Exibe o formulário de forma modal
        Form2->ShowModal();
    }
    __finally
    {
        // Free() verifica se o ponteiro é válido e chama delete internamente
        Form2->Free();
    }
}
```

Código 6

Vale ressaltar que o comando *Free* foi adicionado no mesmo botão por motivo do *ShowModal*. O comando *ShowModal* pausa o fluxo de instruções até que o formulário chamado seja fechado, ou seja, o comando *Free* só será lido quando o *Form2* for fechado pelo usuário.

Por outro lado, o comando *Show* não pausa o fluxo de instruções. Portanto, se o comando *Show* for utilizado ao invés do *ShowModal* para exibir o formulário, então o comando *Free* terá que ser inserido no evento *OnClose* do *Form2*. Em outras palavras, o *Form2* só deverá ser liberado da memória após ser fechado pelo usuário.

Mesmo com a opção acima, o C++Builder ainda disponibiliza uma forma de criar novos formulários em tempo de execução, sem a necessidade de que estes estejam disponíveis em tempo de projeto.

Para realizar este teste, adicione outro botão ao *Form1* e altere a propriedade *Caption* para “Criar Formulário”. O código a seguir cria o *Form3* em **tempo de execução e atribui algumas propriedades antes de ser exibido ao formulário**:

```

40 //-----
void __fastcall TForm1::Button2Click(TObject *Sender)
{
    TForm *Form3 = new TForm(this); // cria e instancia o formulário
    try
    {
        Form3->Caption = "Novo Formulário";
        Form3->BorderStyle = bsDialog;
        Form3->Position = poScreenCenter;
        Form3->ShowModal(); // exibe o formulário de forma modal
50    }
    finally
    {
        delete Form3; // libera o formulário da memória
54    }
}
//-----

```

Código 7

A Figura 9 demonstra a função do botão acima quando a aplicação for executada:

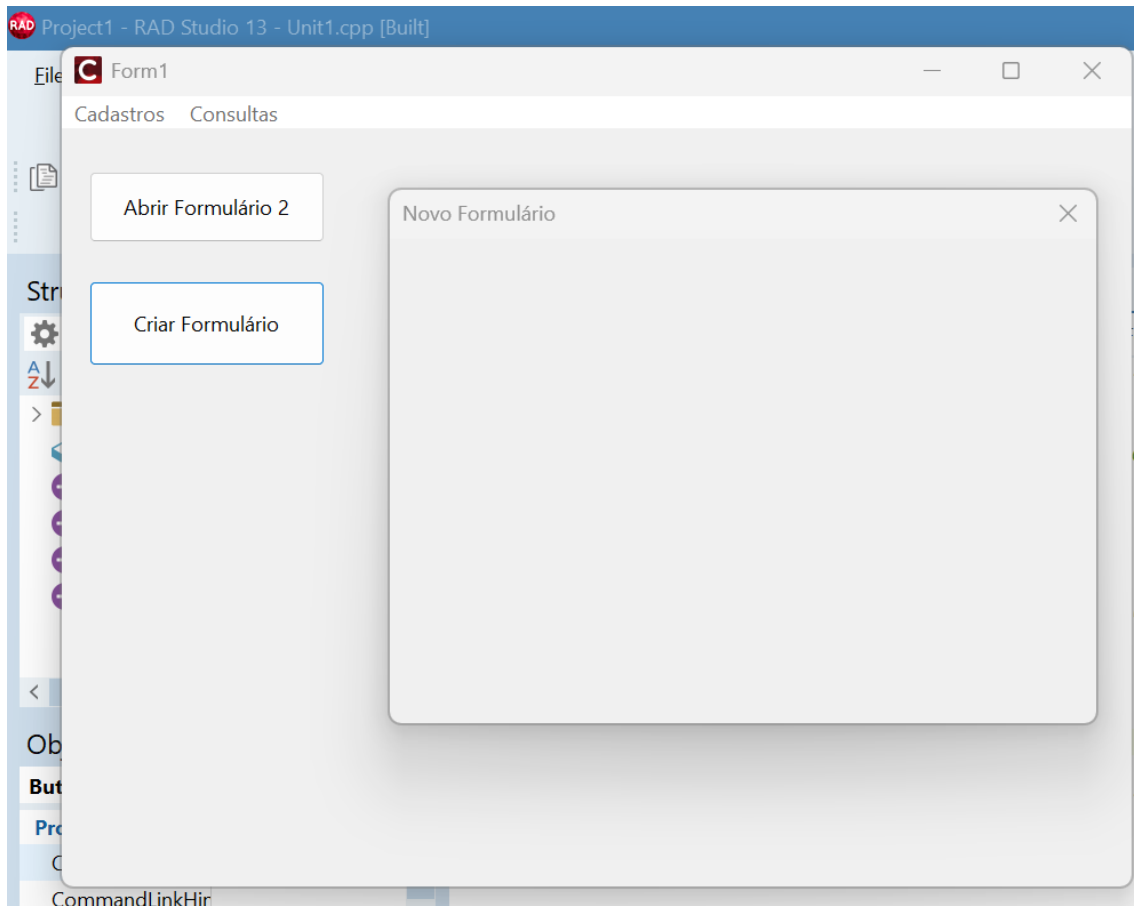


Figura 9: Criando um novo formulário em tempo de execução

CAIXAS DE DIÁLOGO

Mensagens na tela são uma forma intuitiva de informar ou avisar o usuário sobre determinado evento ocorrido na aplicação. Além disso, o desenvolvedor também pode criar caixas de diálogo para confirmar uma ação do usuário e executar um conjunto específico de instruções.

No C++Builder é possível criar caixas de diálogo simples e avançadas, dependendo de necessidade do desenvolvedor. A forma mais simples de mostrar uma mensagem ao usuário pode ser realizada através do comando *ShowMessage*, conforme no código abaixo:

```

• //-----
• void __fastcall TForm1::Button1Click(TObject *Sender)
• {
30  ShowMessage("Mensagem de exemplo");
• }
• //-----

```

Código 8

Porém, o *ShowMessage* não oferece opções adicionais, como inserir um ícone na caixa de diálogo ou adicionar botões de decisão do usuário. Para exibir uma caixa de

diálogo com recursos mais avançados, deve-se utilizar o *MessageDlg* e conhecer os parâmetros exigidos na sua sintaxe:

```
MessageDlg(
    "Mensagem",          // Texto exibido na caixa de diálogo
    mtInformation,        // Tipo da mensagem (ícone e estilo)
    TMsgDlgButtons() << mbOK, // Botões disponíveis para o usuário
    0                    // Índice de ajuda (geralmente 0 se não usar)
);
```

O exemplo abaixo exibe uma caixa de diálogo simples com um ícone de informação:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    MessageDlg("Olá", mtInformation, TMsgDlgButtons() << mbOK, 0);
}
//-----
```

Código 9

Explicando....

“Olá” é a mensagem a ser exibida;

mtInformation é o tipo de ícone da mensagem. Existem outros dentro da chamada *MessageDlg* como *mtWarning*, *mtError*, *mtConfirmation*, *mtInformation*, *mtCustom* (conforme sua necessidade).

TMsgDlgButtons é uma classe/struct que representa um conjunto de botoes da mensagem (*TMsgDlgButton*);

() cria um conjunto vazio inicialmente;

<< **mbOK** adiciona o botão OK ao conjunto.

Quando dois ou mais botões forem adicionados ao parâmetro do *MessageDlg*, é preciso que a resposta do usuário seja verificada através de uma condição “if”. O exemplo a seguir exibe uma mensagem de confirmação com os botões “Sim” e “Não”, com a instrução executada de acordo com a resposta do usuário:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    if (MessageDlg("Deseja fechar o formulário?", mtConfirmation,
        TMsgDlgButtons() << mbYes << mbNo, 0) == mrYes)
    {
        Close();
    }
    else
    {
        MessageDlg("O formulário não será fechado.", mtWarning,
            TMsgDlgButtons() << mbOK, 0);
    }
}
//-----
```

Código 10

Além do *ShowMessage* e *MessageDlg*, o C++Builder ainda disponibiliza o *Application.MessageBox*, que permite a personalização do título da caixa de diálogo,

oferece mais opções de botões e ícones, e utiliza a API do Windows para desenhar a caixa de diálogo. Dessa forma, os botões exibidos na caixa de diálogo aparecerão em Português, ao contrário do *MessageBox*, que exibe os botões no idioma Inglês. Para fazer uma breve comparação entre os dois comandos, sem adicionar funcionalidades aos botões de decisão, digite o código a seguir e execute a aplicação para conferir o estilo de cada mensagem, o título e o idioma dos botões:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    // Caixa de diálogo usando Application->MessageBox
    Application->MessageBox(L"Mensagem.", L"Titulo Personalizado", MB_YESNO | MB_ICONEXCLAMATION);

    // Caixa de diálogo usando MessageDlg
    MessageDlg(L"Mensagem.", mtWarning, TMsgDlgButtons() << mbYes << mbNo, 0);
}
//-----

```

Código 10

Como é possível notar, a forma como o *Application->MessageBox* é declarado é diferente dos parâmetros do *MessageDlg*, apesar de serem bem simples de compreender:

```

Application->MessageBox(
    L"Mensagem",    // Texto exibido na caixa de diálogo
    L"Titulo",      // Título da janela
    MB_YESNO | MB_ICONEXCLAMATION // Botões e ícone combinados
);

```

Os **parâmetros de botões e ícone devem ser unidos pelo operador |** (bitwise OR).

No exemplo acima:

- MB_YESNO → botões “Sim” e “Não”;
- MB_ICONEXCLAMATION → ícone de exclamação padrão do Windows.

Esse conjunto de parâmetros pode ser extenso, portanto é útil usar **CTRL + Espaço** no IDE para abrir a lista de opções disponíveis, conforme a Figura 10:

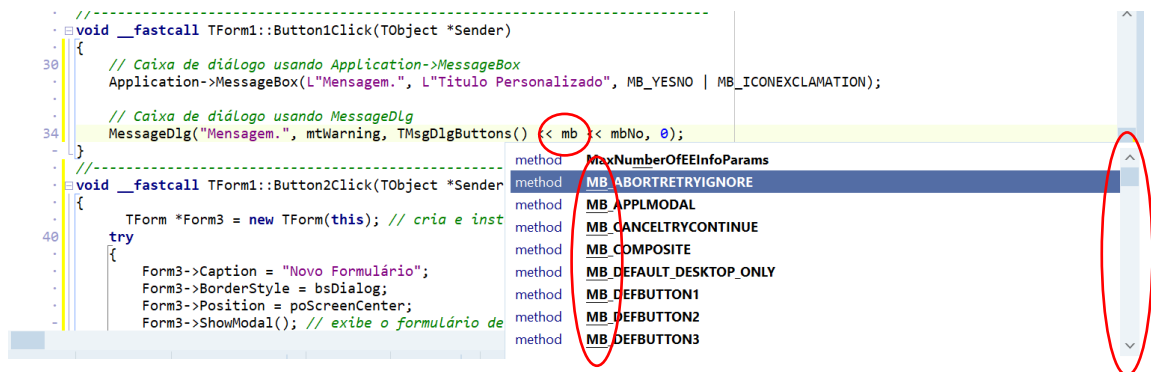


Figura 10: Janela de opções disponíveis para a configuração de botões e ícone

Além das caixas de diálogo para exibir mensagens, o C++Builder também conta com alguns componentes exclusivos de diálogo para interagir com o usuário, disponíveis na paleta *Dialogs*. Crie um novo projeto e arraste o componente *OpenDialog* desta paleta para dentro do formulário. A função deste componente é exibir uma caixa de diálogo para abrir um arquivo. Assim como o componente *MainMenu*, o *OpenDialog* é um componente não-visual e possui algumas propriedades para personalização da caixa de diálogo:

Propriedade *DefaultExt*: define a extensão padrão selecionada na opção “Tipo” da caixa de diálogo.

Propriedade *FileName*: atribui um nome padrão ao campo “Nome” da caixa de diálogo. É uma forma de pré-determinar o nome do arquivo que será selecionado pelo usuário.

Propriedade *Filter*: restringe somente os arquivos com as extensões definidas nessa propriedade. Ao clicar no botão de reticências, um editor é aberto para adicionar somente as extensões que serão permitidas, conforme o exemplo da Figura 11:

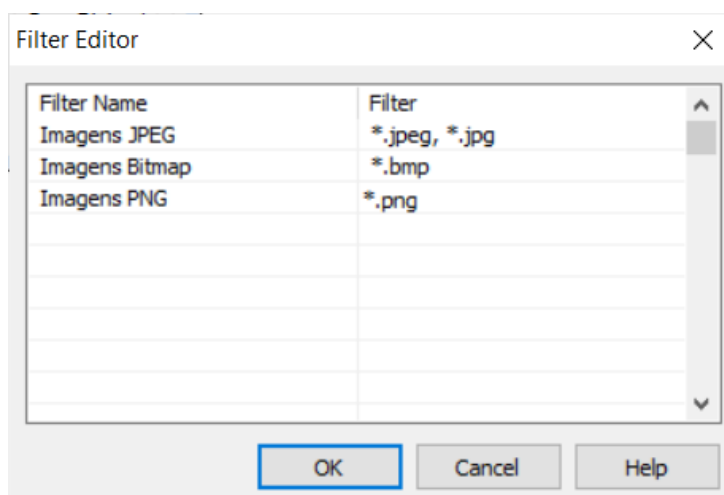


Figura 11: Editor de extensões do componente *OpenDialog*

Propriedade *InitialDir*: determina o diretório padrão que será listado quando a caixa de diálogo for exibida ao usuário.

Propriedade *Title*: define o texto que será exibido na barra de título da caixa de diálogo.

Em tempo de execução, é necessário digitar o comando *Execute* para abrir a caixa de diálogo ao usuário. Se este comando retornar um valor *True*, significa que o usuário selecionou um arquivo válido. Para exemplificar, adicione um botão ao formulário, um componente *Memo*, e escreva o seguinte código:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    OpenDialog1->Filter = "TXT|*.txt"; // restringe a extensão para .txt
    if (OpenDialog1->Execute())
    {
        Memo1->Lines->LoadFromFile(OpenDialog1->FileName); // carrega o conteúdo
    }
}
//-----
```

Código 10

Execute a aplicação e selecione qualquer arquivo de texto (.txt) no computador. Ao pressionar o botão OK, o conteúdo do arquivo é atribuído automaticamente ao componente *Memo*, conforme demonstrado na Figura 12:

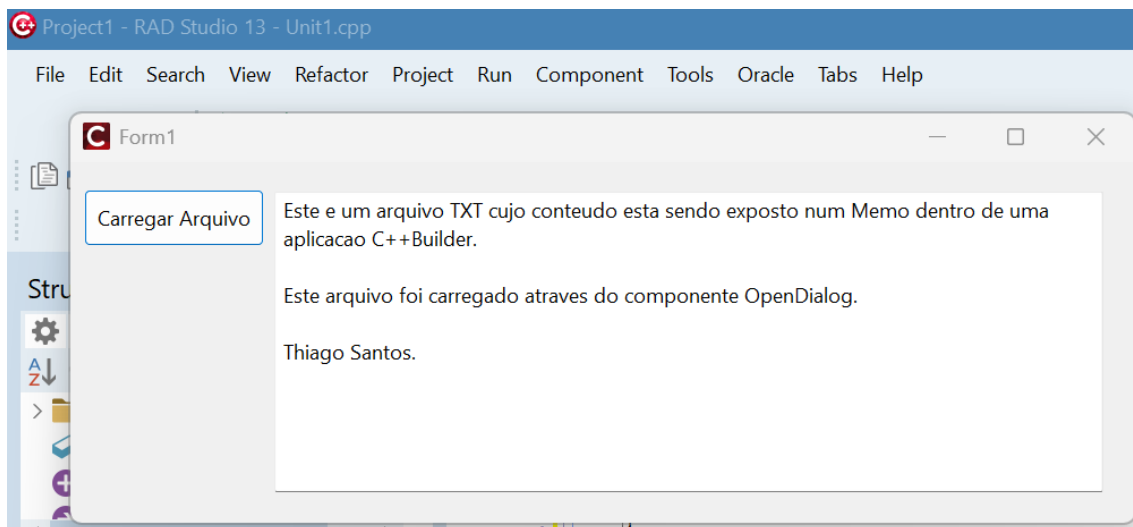


Figura 12: Conteúdo de um arquivo texto carregado dentro do componente Memo

Ainda na paleta de componentes *Dialogs*, o componente *SaveDialog* realiza justamente o inverso do *OpenDialog*, ou seja, abre uma caixa de diálogo para salvar um arquivo. Mesmo com este diferencial, o componente *SaveDialog* tem basicamente as mesmas propriedades do *OpenDialog*.

Adicione um componente *SaveDialog* ao formulário e substitua o código do botão para:

```
//-----
1 void __fastcall TForm1::Button1Click(TObject *Sender)
2 {
3     SaveDialog1->DefaultExt = ".txt"; // restringe a extensão para .txt
20
3     if (SaveDialog1->Execute())
4     {
5         Memo1->Lines->SaveToFile(SaveDialog1->FileName); // salva o conteúdo
24
6     }
7 }
8 //-----
```

Código 11

Execute a aplicação novamente, mas desta vez escreva algo dentro do componente *Memo* e então clique no botão. A caixa de diálogo será exibida para que o usuário selecione o diretório e digite um nome para o arquivo que será salvo. A Figura 13 exemplifica um conteúdo digitado no *Memo* e a uma imagem do arquivo aberto pelo Bloco de Notas após ser salvo:

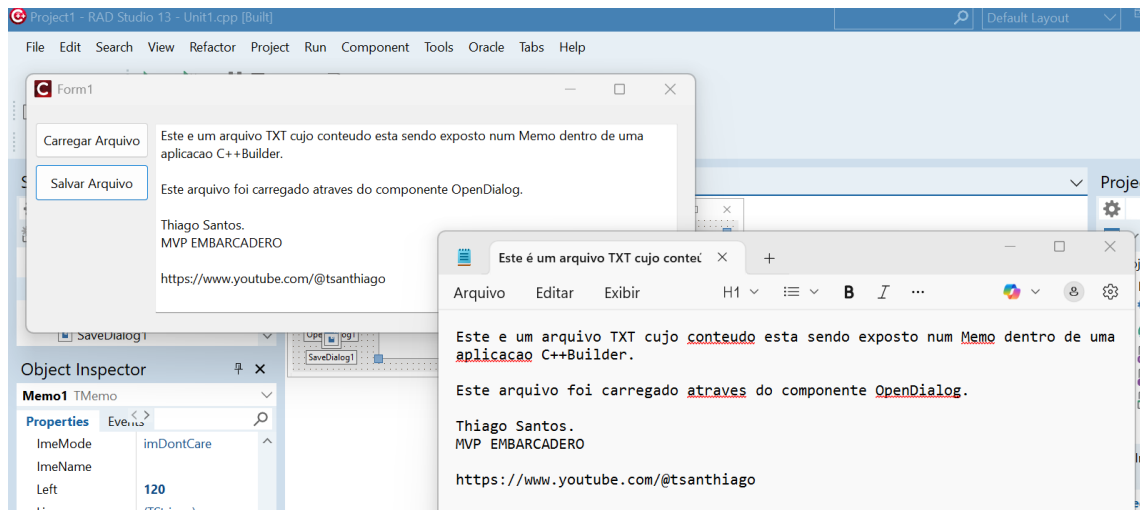


Figura 13: Exemplo de um arquivo salvo através do SaveDialog

HERANÇA DE FORMULÁRIOS

Uma das principais características de uma boa aplicação é a **padronização dos formulários utilizados pelo usuário**. Essa padronização garante uma identidade visual consistente para a aplicação, além de economizar tempo de desenvolvimento e facilitar a manutenção do sistema.

Por esse motivo, muitos desenvolvedores adotam a prática de **criar um formulário base**, contendo componentes visuais, comportamentos e métodos comuns, e **herdar essas características em outros formulários** criados posteriormente.

O conceito de **herança** é amplamente utilizado na **Programação Orientada a Objetos**, e no **C++Builder** ele pode ser aplicado diretamente aos formulários, já que todo formulário é uma classe derivada de **TForm**.

Para herdar formulários no C++Builder, o primeiro passo é criar um **formulário padrão (formulário base)**, que servirá como modelo. Os demais formulários irão herdar desse formulário base, reutilizando sua estrutura visual, propriedades e métodos, garantindo assim maior organização, reaproveitamento de código e padronização da aplicação.

A Figura 14 exibe um exemplo de formulário padrão, com 1 *Label* na parte superior, 6 *BitBtn*, 1 *Panel* e 1 *StatusBar*. O objetivo é manter este visual com os componentes adicionados para os formulários herdados.

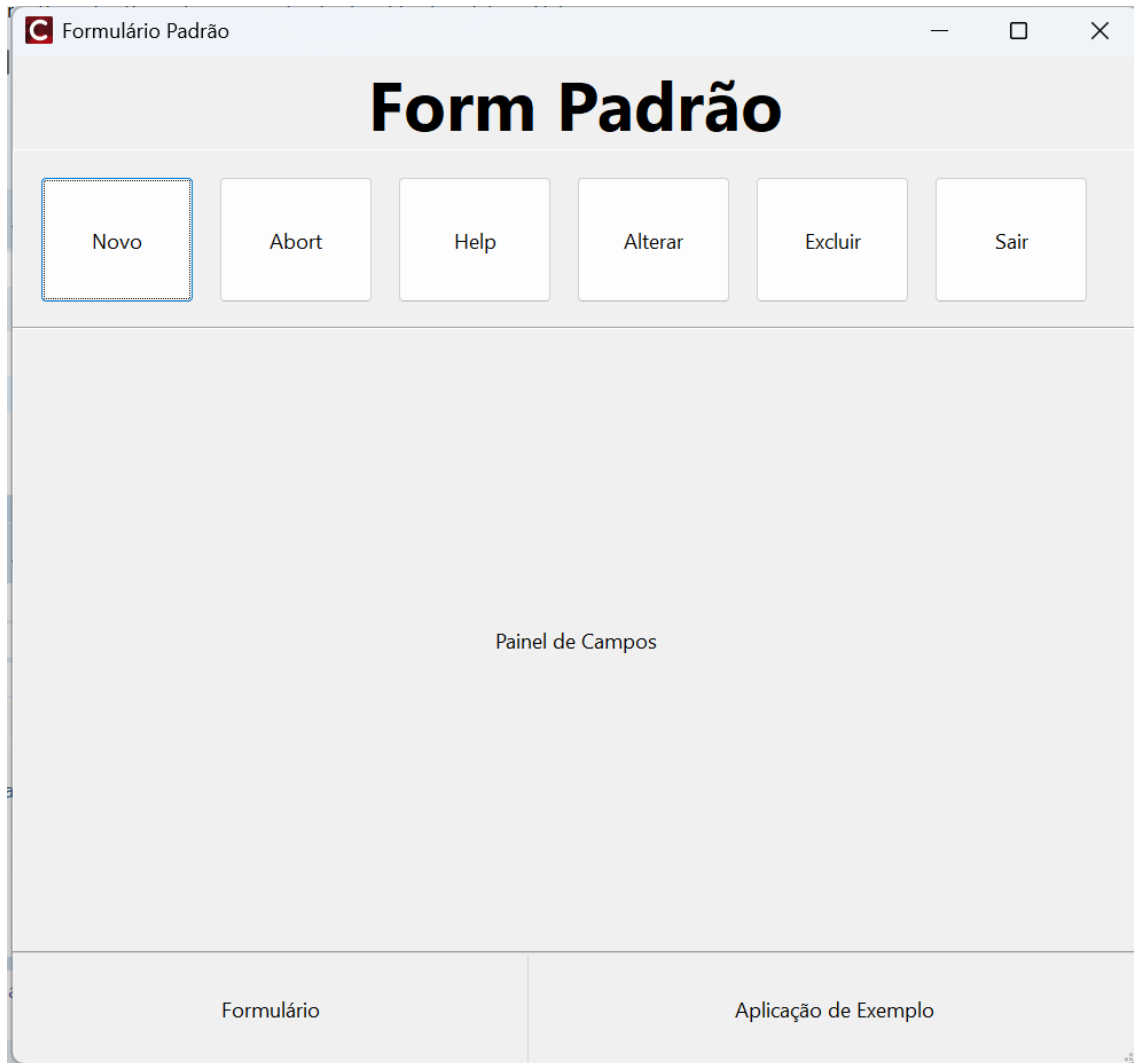


Figura 14: Criando um formulário padrão em um projeto

Crie um novo projeto e modele o formulário com o visual semelhante ao exemplo acima. Renomeie cada botão com a sua respectiva função: *btnNovo*, *btnGravar*, *btnCancelar*, *btnAlterar*, *btnExcluir* e *btnSair*. Para exemplificar a herança de métodos, escreva os seguintes códigos para o botão “Novo” e “Sair”:

```
//-----
void __fastcall TForm1::btnNovoClick(TObject *Sender)
{
    ShowMessage("O botão novo foi clicado !");
}
//-----
void __fastcall TForm1::btnSairClick(TObject *Sender)
{
    Close();
}
//-----
```

Código 12

Depois de modelar o formulário da forma desejada, altere a propriedade *Name* do formulário para “Form_Heranca”, e salve-o com o nome “Unit_Heranca” na pasta do projeto. É importante salientar que este formulário somente será utilizado para fins de herança. Portanto, quaisquer outras alterações particulares deverão ser realizadas nos formulários herdados.

A herança de formulários é realizada de forma bem simples no C++Builder. Basta acessar o menu *File > New > Other* e procurar pelo item *Inheritable Items*, que consiste em uma lista de todos os itens no projeto que podem ser herdados. Veja que o formulário *Form_Heranca* já aparece do lado direito, conforme a Figura 15:

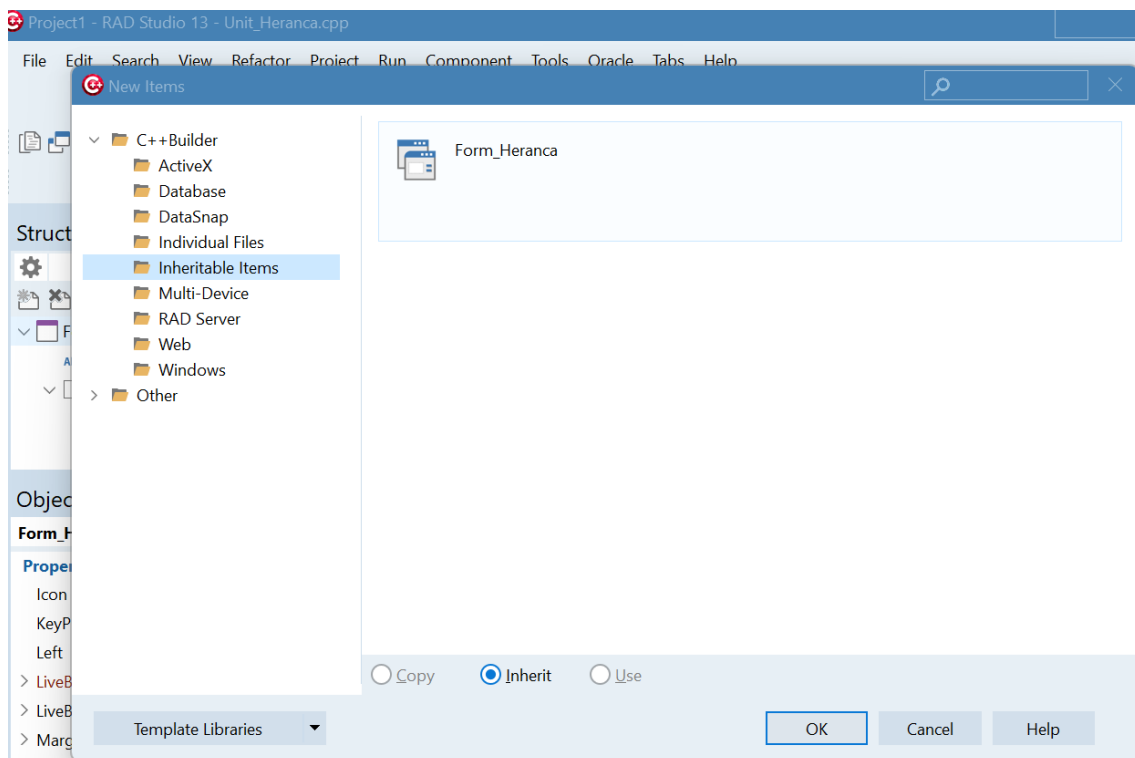


Figura 15: Acessando a janela de itens que podem ser utilizados para herança

Basta somente selecioná-lo e clicar em OK para que um formulário herdado do *Form_Heranca* seja criado e exibido na tela. Por padrão, o C++Builder enumera novos formulários em ordem numérica, neste caso atribuindo o nome *UnitX.cpp* ao formulário herdado. Cabe ao desenvolvedor atribuir um nome mais sugestivo em relação ao objetivo do formulário, como *Form_Clientes*, por exemplo.

Agora note que, ao clicar duas vezes no botão no botão “Novo”, ele aparece com o nome Heranca2:

```

18 //
19 void __fastcall TForm_Heranca2::btnNovoClick(TObject *Sender)
20 {
21 }
22 //

```

Código 13

Isso significa que os métodos inseridos no botão “Novo” do formulário *Form_Heranca* também serão herdados. Portanto, ao executar a aplicação e clicar no botão “Novo” no formulário herdado, a mensagem será exibida mesmo que não tenha um *ShowMessage* no botão. Dessa forma, torna-se muito mais fácil definir um conjunto de instruções padrão sem a necessidade de reescrever o código para cada botão.

Outra vantagem da herança é a simplicidade na manutenção de formulários. No exemplo acima, caso seja necessário fazer uma alteração nos formulários herdados (como adicionar um novo botão, por exemplo), basta somente alterar o *Form_Heranca* para que todos os formulários também recebam essa alteração.